Laboratorio de Programación y Estructuras de Datos (PRD) – Sesión 6: Gestión dinámica de memoria en C

En esta sesión trabajaremos la gestión dinámica de memoria en C mediante el uso de las funciones de librería `malloc()`, `calloc()`, `realloc()` y `free()` definidas en `stdlib.h` para este propósito. También introduciremos la herramienta Valgrind para el análisis y testeo del uso de memoria de nuestro programa en C.

1. Desarrollo de la práctica

En esta práctica se propone la implementación de un registro de números de identificación fiscal (NIFs) mediante una estructura de datos dinámica de tipo lista basada en array, a la que nos referiremos por su nombre en inglés *array list*. Esta estructura de datos será capaz en todo momento de adaptar su capacidad dinámicamente en función del número de NIFs que guardemos en el registro.

Ejercicio 1:

Complete el programa en C del fichero *nif.c* proporcionado con el enunciado junto con el fichero de cabeceras *nif.h*, donde se ha definido el siguiente struct:

```
typedef struct {
      int numero;
      char letra;
}t_nif;
```

Para conseguir este objetivo, siga los siguientes pasos:

a) Escriba el código de la función:

```
t_nif generar_nif_aleatorio();
```

Esta función debe generar un NIF aleatoriamente, depositarlo en una nueva variable de tipo `t_nif` y devolverlo. Para ello, generará un valor aleatorio entero de 8 dígitos, comprendido en el rango entre [10000000, 99999999], que representará el número del nuevo NIF. Además, mediante el módulo (%) 23 del número de NIF, se determinará su letra según la siguiente tabla:

Resultado	0	1	2	3	4	5	6	7	8	9	10	11
Letra	T	R	W	A	G	M	Y	F	P	D	X	B
Resultado	12	13	14	15	16	17	18	19	20	21	22	-
Letra	N	J	Z	S	Q	V	H	L	C	K	E	-

Para la generación de valores enteros aleatorios en C tenga en cuenta la función `rand()` definida en la librería `stdlib.h`. Esta función devuelve un valor entero dentro del rango entre [0, RAND_MAX], siendo RAND_MAX una constante simbólica definida en la misma librería con valor muy grande. Así, los valores que devuelva `rand()` deberán acotarse al rango deseado. **Pista**: utilice el operador módulo.

b) Escriba el código de la función:

```
void mostrar_nif(t_nif nif);
```

Esta función muestra por pantalla los valores de los campos (número y letra) de la variable de tipo `t_nif` pasada como parámetro.

c) Escriba el código de la función:

```
int comparar_nifs(t_nif nif1, t_nif nif2);
```

Esta función devuelve un valor entero >0 si el número de `nif1` es mayor al de `nif2`, un valor entero <0 si el número de `nif1` es menor al de `nif2`, o 0 si los números de `nif1` y `nif2` son el mismo.

Ahora escriba el código de la función `void sesion6_ej1()` en el fichero *ejercicios.c* proporcionado con el enunciado junto al fichero de cabeceras *ejercicios.h*. Esta función genera 2 NIFs aleatoriamente, los muestra por pantalla e indica cuál de los dos es mayor (o si son iguales). Esta función deberá llamarse desde el fichero *main.c*, testeando así el correcto funcionamiento de las funciones codificadas anteriormente.

Nota: Como podrá comprobar, los NIFs generados aleatoriamente son siempre los mismos a lo largo de las distintas ejecuciones del programa. Esto sucede porque la semilla que toma en cuenta `rand()` por defecto para la generación de valores aleatorios siempre es la misma. Es por ello que normalmente se intenta configurar una semilla distinta en cada ejecución del programa, consiguiendo así que `rand()` genere valores aleatorios distintos. Esto se consigue mediante la función `srand()`, que se invoca **sólo una vez** al principio de la ejecución del programa. Una estrategia muy utilizada es proporcionar el tiempo actual del computador como semilla (parámetro) a `srand()` que podemos obtener mediante la instrucción `time(NULL)` siendo `time()` una función definida en la librería `time.h`. Para comprobarlo, importe la librería `time.h` en el fichero *main.c*, e incluya la instrucción `srand(time(NULL));` al principio de la función `main()`.

Ejercicio 2:

Implemente en el fichero *array_list.c* proporcionado con el enunciado algunas de las funciones definidas en el fichero de cabeceras *array_list.h*, donde se define la siguiente estructura:

```
typedef struct {
      int capacidad;
      int ocupada;
      t_nif *nifs;   /*Array interno de variables t_nif*/
}t_array_list;
```

Implemente las funciones que se solicitan en los siguientes apartados.

a) Escriba el código de la función:

```
t_array_list * crear_lista();
```

Esta función debe efectuar una reserva de memoria dinámica mediante la función `malloc()` para almacenar una nueva variable de tipo `t_array_list`. A su vez, mediante la función `calloc()` también reservará memoria dinámica para almacenar el array interno de variables de tipo `t_nif` de CAPACIDAD_INICIAL posiciones (constante simbólica definida en *array_list.h* con valor igual a 5). La función devuelve un puntero al bloque de memoria dinámica reservado para la lista, o NULL si ha existido cualquier problema en la reserva de memoria.

b) Escriba el código de la función:

```
void liberar_lista(t_array_list *lista);
```

Esta función debe liberar todo bloque de memoria dinámica previamente reservado para almacenar la `t_array_list` a la que apunta `lista`.

c) Escriba ahora el código de la función:

```
int add(t_array_list *lista, t_nif nif);
```

Esta función debe añadir el NIF proporcionado como parámetro al final de la *array list* apuntada por `lista`. Si en el momento de añadir el NIF la capacidad ocupada de la *array list* ya alcanza su capacidad total, se realizará un redimensionado de su array interno multiplicando x2 su capacidad total mediante la función `realloc()`. En caso que exista cualquier problema en la reserva de memoria durante este proceso, la función devuelve -1. Si hay capacidad suficiente en el array interno para almacenar `nif`, este se añade en su primera posición vacía (no ocupada aún), actualizando después la capacidad ocupada de la lista. Finalmente, la función devuelve 0.

d) Escriba ahora el código de la función:

```
void mostrar_lista(t_array_list *lista);
```

Esta función debe mostrar el contenido de la *array list* apuntada por `lista`. Es decir, debe mostrar la capacidad total y ocupada de la lista, además del número y la letra de cada uno de los NIFs almacenados en las distintas posiciones de su array interno. Puede utilizar la función implementada en el ejercicio 1.b para este propósito.

Finalmente, escriba el código de la función `void sesion6_ej2()` en el fichero *ejercicios.c* proporcionado con el enunciado de la práctica. En esta función debe crear una nueva *array list*, rellenándola inicialmente con 7 NIFs generados de forma aleatoria mediante la función implementada en el ejercicio 1.a. Después, debe mostrar el contenido de la lista. Acto seguido, añada 4 NIFs aleatorios más y vuelva a mostrar el contenido de la lista. Para acabar, libere cualquier bloque de memoria dinámica previamente reservado. Esta función deberá llamarse desde el fichero *main.c*, testeando así el correcto funcionamiento de las funciones codificadas anteriormente.

Ejemplo de ejecución:

```
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G

Mostrando contenido de lista (capacidad ocupada: 11/20)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G
39760510L
66516655B
29641434T
45202373J
```

Cuando trabaje con memoria dinámica en C, debe asegurarse siempre que cualquier bloque de memoria dinámica reservado se libera adecuadamente cuando ya no sea

necesario. Sino, se producirán fugas de memoria (*memory leaks*), limitando el rendimiento del programa. En programas complejos, el control absoluto de fugas de memoria no es sencillo. Por ello, existen aplicaciones que analizan el uso de memoria dinámica de un programa en C, informando de cualquier fuga de memoria que pueda haberse producido. Una de estas aplicaciones es Valgrind (http://valgrind.org/), instalada en los ordenadores del aula de laboratorio de PRD.

El uso básico de Valgrind es sencillo: desde un terminal, p.ej., el terminal integrado en el IDE Netbeans, debe ir al directorio donde está almacenado el binario del programa resultante de las fases de compilación y enlazado. En los ordenadores del aula de laboratorio, desde la carpeta de su proyecto Netbeans debe escribir: `cd /dist/Debug/GNU-Linux`. Luego, ejecute: `valgrind ./nombre`, donde `nombre` es el nombre del binario del programa (que puede conocer mediante el comando `ls` de Linux, que le mostrará los nombres de los ficheros del directorio).

Ejemplo de testeo, donde se remarca en negrita el resumen del testeo de uso de memoria del programa anterior, mostrando que no se ha producido ninguna fuga de memoria:

```
valgrind ./prd_-_sesion_6
==13304== Memcheck, a memory error detector
==13304== Copyright (C) 2002-2015, and GNU GPL'd, by Julian Seward et al.
==13304== Using Valgrind-3.11.0 and LibVEX; rerun with -h for copyright
info
==13304== Command: ./prd_-_sesion_6
==13304==
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G
Mostrando contenido de lista (capacidad ocupada: 11/20)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G
39760510L
66516655B
29641434T
45202373J
==13304==
==13304== HEAP SUMMARY:
==13304==     in use at exit: 0 bytes in 0 blocks
==13304==   total heap usage: 5 allocs, 5 frees, 1,320 bytes allocated
==13304==
==13304== All heap blocks were freed -- no leaks are possible
==13304==
==13304== For counts of detected and suppressed errors, rerun with: -v
==13304== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Este resultado hubiera sido distinto si no se hubieran liberado, al final de la función `sesion6_ej2()` todos los bloques de memoria previamente reservados para almacenar la *array list* creada. En este caso, por ejemplo, se reportan dos fugas de memoria, causadas por la no liberación de un bloque de 16 Bytes y otro de 160 Bytes. El primero de ellos es el bloque de memoria dinámica reservado para almacenar la variable de tipo

`t_array_list`, compuesto por dos campos de tipo `int` (4 + 4 Bytes) más un puntero, que ocupa un registro de 8 Bytes en una CPU de 64 bits. Y el otro bloque de memoria es el reservado para almacenar el vector interno de variables `t_nif` de la lista, de 20 posiciones en el momento de finalizar el programa. Aunque el struct `t_nif` define dos campos, un `int` (4 Bytes) y un `char` (1 Byte), el compilador acaba reservando 4 Bytes para almacenar el `char` del struct, para optimizar la alineación de los datos en memoria. De ahí, 20*(4 + 4 Bytes) = 160 Bytes.

```
valgrind ./prd_-_sesion_6
==13396== Memcheck, a memory error detector
==13396== Copyright (C) 2002-2015, and GNU GPL'd, by Julian Seward et al.
==13396== Using Valgrind-3.11.0 and LibVEX; rerun with -h for copyright
info
==13396== Command: ./prd_-_sesion_6
==13396==
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G
Mostrando contenido de lista (capacidad ocupada: 11/20)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G
39760510L
66516655B
29641434T
45202373J
==13396==
==13396== HEAP SUMMARY:
==13396==     in use at exit: 176 bytes in 2 blocks
==13396==   total heap usage: 5 allocs, 3 frees, 1,320 bytes allocated
==13396==
==13396== LEAK SUMMARY:
==13396==    definitely lost: 16 bytes in 1 blocks
==13396==    indirectly lost: 160 bytes in 1 blocks
==13396==      possibly lost: 0 bytes in 0 blocks
==13396==    still reachable: 0 bytes in 0 blocks
==13396==         suppressed: 0 bytes in 0 blocks
==13396== Rerun with --leak-check=full to see details of leaked memory
==13396==
==13396== For counts of detected and suppressed errors, rerun with: -v
==13396== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

A partir de este momento, asegúrese mediante el uso de Valgrind que en sus programas en C no se originan fugas de memoria cuando utilice memoria dinámica.

Ejercicio 3:

Implemente en el fichero *array_list.c* proporcionado con el enunciado algunas funciones más, también definidas en el fichero de cabeceras *array_list.h*.

a) Escriba el código de la función:

```
void ordenar_lista_por_nif(t_array_list *lista);
```

Esta función debe ordenar el contenido del array interno de la lista apuntada por `lista`, es decir ordenando los NIFs almacenados en orden creciente, según su número. Implemente el algoritmo de ordenación de vectores por inserción (Insertion Sort) presentado en las sesiones de teoría de la asignatura para este propósito. Utilice la función implementada en el ejercicio 1.c para comparar los distintos NIFs.

Ahora escriba el código de la función `void sesion6_ej3a()` en el fichero *ejercicios.c* proporcionado con el enunciado de la práctica. En esta función debe crear una nueva *array list*, rellenándola inicialmente con 7 NIFs generados de forma aleatoria y mostrar su contenido. Acto seguido, utilice la función `ordenar_lista_por_nif()` para ordenar la lista y vuelva a mostrar su contenido. Esta función deberá llamarse desde el fichero *main.c*, testeando así el correcto funcionamiento de la función codificada anteriormente.

Ejemplo de ejecución:

```
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
46930895Q
71692795R
14636934X
77747814V
74238339C
99885393G

Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
14636934X
46930895Q
71692795R
74238339C
77747814V
99885393G
```

b) Escriba el código de la función:

```
int insertar_en_orden(t_array_list *lista, t_nif nif);
```

Esta función debe insertar el NIF proporcionado como parámetro en la *array list* apuntada por `lista`, que se asume ordenada, guardando los distintos NIFs en orden creciente según su número. El nuevo NIF debe insertarse en aquella posición tal que se siga manteniendo dicho orden.

Ahora escriba el código de la función `void sesion6_ej3b()` en el fichero *ejercicios.c* proporcionado con el enunciado de la práctica. En esta función debe crear una nueva *array list*, rellenándola inicialmente con 7 NIFs generados de forma aleatoria, todos ellos mediante la función `insertar_en_orden()`, y mostrar su contenido. En este caso, el programa debería mostrar directamente la lista ordenada. Esta función deberá llamarse desde el fichero *main.c*.

c) Escriba el código de la función:

```
int buscar_registro_ordenado(t_array_list *lista, t_nif nif);
```

Esta función debe buscar si el NIF proporcionado como parámetro está guardado en la *array list* a la que apunta `lista`, que se asume ordenada de forma creciente. La función debe aprovechar este orden para realizar una búsqueda binaria, tal y como se ha

explicado en las sesiones de teoría de la asignatura. La función devuelve el índice (posición) en el que se ha encontrado el NIF buscado, o -1 si este no existe en la lista.

Escriba el código de la función `void sesion6_ej3c()` en el fichero *ejercicios.c* proporcionado con el enunciado de la práctica. En esta función debe crear una nueva *array list*, rellenándola inicialmente con 7 NIFs generados de forma aleatoria, todos ellos mediante la función `insertar_en_orden()` y mostrar su contenido. Luego deberá solicitar la introducción de un NIF a buscar, e informar la posición en la que ha sido encontrado mediante `buscar_registro_ordenado()`, o si no lo ha sido. Esta función deberá llamarse desde el fichero *main.c*.

Ejecuciones de ejemplo (en negrita los datos introducidos por el usuario):

```
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
14636934X
46930895Q
71692795R
74238339C
77747814V
99885393G

Introduce el nif a buscar: 74238339C
Nif encontrado en la posicion 4

--------------

Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
14636934X
46930895Q
71692795R
74238339C
77747814V
99885393G

Introduce el nif a buscar: 11111111A
Nif no encontrado...
```

Ejercicio 4 (adicional):

Implemente en el fichero *array_list.c* proporcionado con el enunciado la función:

```
int eliminar_nif(t_array_list *lista, t_nif nif);
```

Esta función debe eliminar el NIF proporcionado como parámetro de la *array list* apuntada por `lista`, en caso que exista. Asuma que los NIFs están ordenados de forma creciente según su número en ella, lo que le permite buscar el NIF a eliminar mediante una búsqueda binaria. Si el NIF ha sido encontrado, se procederá a actualizar su capacidad ocupada (`lista->ocupada--`). En caso que la capacidad ocupada de la lista pase a ser inferior a ¼ de su capacidad total, se redimensionará su array interno dividiendo entre 2 su capacidad total mediante la función `realloc()` de `stdlib.h`. La función devolverá 1 si el NIF ha sido encontrado y eliminado satisfactoriamente, 0 si el NIF no ha sido encontrado o -1 si ha ocurrido cualquier problema durante el redimensionado del array interno.

Escriba el código de la función `void sesion6_ej4()` en el fichero *ejercicios.c* proporcionado con el enunciado de la práctica. En esta función debe crear una nueva *array list*, rellenándola inicialmente con 7 NIFs generados de forma aleatoria y mostrar su contenido. Luego deberá solicitar la introducción de un NIF a eliminar utilizando la función `eliminar_nif()`. El programa deberá informar si el NIF ha sido eliminado o no, o si ha sucedido cualquier problema durante el redimensionado de la lista. Además, deberá volver a mostrar el nuevo contenido de la lista por pantalla. Esta función deberá llamarse desde el fichero *main.c*.

Ejecuciones de ejemplo (en negrita los datos introducidos por el usuario):

```
Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
14636934X
46930895Q
71692795R
74238339C
77747814V
99885393G

Introduce el nif a eliminar: 46930895Q
Nif eliminado.

Mostrando contenido de lista (capacidad ocupada: 6/10)
14289403D
14636934X
71692795R
74238339C
77747814V
99885393G

-----------------------------

Mostrando contenido de lista (capacidad ocupada: 7/10)
14289403D
14636934X
46930895Q
71692795R
74238339C
77747814V
99885393G

Introduce el nif a eliminar: 11111111A
Nif no encontrado...
```